## Strengths and Limitations of Nearest-Neighbor Classification

Before applying KNN to real data, it is worth taking stock of what the method buys you and what it costs. On the plus side, learning is fast — in fact there is no explicit training at all. KNN is a *lazy learner*: "training" amounts to nothing more than storing the examples, so there is no optimization step to run. Second, no theory is required: you do not need a model of the underlying process that generated the data, only the data itself and a sensible distance metric. Third, the method is easy to explain, both in its mechanics and its results — "we classified this example like its nearest neighbors" is a story anyone can follow.

The disadvantages mirror these virtues. The method is memory intensive, because every example must be kept around indefinitely. Predictions can take a long time, because brute-force nearest neighbor compares a new example against *every* stored one; better algorithms than brute force exist for finding neighbors, so the naive implementation is not the last word. Most philosophically important: KNN gives you no model that sheds light on the process that generated the data. It will happily predict, but it does not explain — it never hands you a relationship between variables that you can reason about. This tension between predictive power and explanatory power motivates the evaluation machinery of the following sections: if the model cannot explain itself, we must at least measure it rigorously.

## Case Study: Predicting Survival on the Titanic

The lecture turns to a somber real dataset: the RMS Titanic, which sank in the North Atlantic on the morning of April 15, 1912 after colliding with an iceberg. Of the 1,300 passengers aboard, 812 died; separately, 703 of the 918 crew members died. The database contains 1,046 passengers, and for each one exactly three features are known: **cabin class** (first, second, or third), **age**, and **gender**.

The natural question is whether KNN can use those three features to predict who lived and who died — a binary classification task. But the professor emphasizes a deeper question hiding inside this exercise: once such a classifier is built, *how do we judge whether it is any good?* That question drives everything that follows, because the obvious answer — accuracy — turns out to be treacherous.

## Why Accuracy Alone Fails on Unbalanced Classes

The sobering baseline: if we predict "died" for every single person, ignoring the features entirely, our accuracy exceeds **62%** for passengers and **76%** for crew members — just by always answering "died." The point becomes even sharper with a disease occurring in 0.1% of the population: a classifier that predicts "disease-free" for everyone achieves accuracy

$$\text{accuracy} = 0.999,$$

i.e., 99.9%, and yet it never identifies a single sick person. When classes are unbalanced, accuracy alone can make a completely useless classifier look excellent. This is precisely the phenomenon the diagnostic-testing literature warns about: a test that always returns a negative result has perfect scores on the metric that counts correct rejections, because that metric simply ignores the sick patients it missed — rendering it useless for detecting the condition. We therefore need metrics built from more granular counts.

## The Four Counts and Their Ratios: Sensitivity, Specificity, PPV, NPV

All alternative metrics are built from four counts produced by comparing predictions against actual outcomes:

- **True positive (TP)**: actually positive, called positive.
- **False positive (FP)**: actually negative, called positive.
- **True negative (TN)**: actually negative, called negative.
- **False negative (FN)**: actually positive, but missed.

From these come four ratios, each answering a different question:

$$\text{sensitivity} = \frac{TP}{TP + FN}, \qquad \text{specificity} = \frac{TN}{TN + FP},$$

$$\text{PPV} = \frac{TP}{TP + FP}, \qquad \text{NPV} = \frac{TN}{TN + FN}.$$

**Sensitivity** asks: of all the examples that really are positive, what fraction did we catch? (In clinical settings it is also called the *detection rate*.) **Specificity** asks: of all the truly negative examples, what fraction did we correctly clear? **Positive predictive value** asks: of everything we called positive, how many really were? **Negative predictive value** asks: of everything we called negative, how many really were?

The diagnostic-testing literature adds crucial interpretation. A test with high sensitivity rarely misdiagnoses those who have the condition, so a *negative* result is useful for **ruling out** disease; a test with 100% sensitivity recognizes every patient with the disease. Conversely, high specificity means the test rarely flags healthy people, so a *positive* result is useful for **ruling in** disease. Each metric is blind to the other error type: a "bogus" test kit designed to always read positive achieves 100% sensitivity while being useless for ruling anything in, and symmetrically a test that always reads negative achieves 100% specificity while missing every sick patient. Statistically, higher sensitivity means a lower type II error rate, and higher specificity means a lower type I error rate — and there is usually a trade-off, such that pushing sensitivity up pushes specificity down and vice versa. When true status cannot be known directly, these quantities are defined relative to a *gold standard* test assumed correct; the terms themselves were introduced by American biostatistician Jacob Yerushalmy in 1947.

Now revisit the always-negative disease classifier through this lens: it has beautiful specificity, but its sensitivity is exactly zero — that is the number that exposes it. The lecture closes with a terminology note: sensitivity is the same thing as **recall**, and the slide maps specificity onto **precision**.

## Principled Testing: Leave-One-Out and Repeated Random Subsampling

However we measure performance, we also need a principled way to *test* — and the message of this section is that how you test is part of the model itself. The general framework here is **cross-validation** (also called rotation estimation or out-of-sample testing): any technique that uses different portions of the data to train and test a model on different iterations, in order to estimate how accurately the model will perform on independent data. Its purpose is to flag problems like overfitting and selection bias and to give insight into generalization. One round partitions the sample into complementary subsets, trains on one (the training set), and validates on the other (the testing set); multiple rounds are combined — e.g., averaged — to reduce variability.

Why not just evaluate on the training data? Because that estimate is optimistically biased. In linear regression, if least squares fits $\widehat{y} = a + \beta^T x$ to $n$ observations with $p$ covariates, then — assuming the model is correctly specified — the expected training-set MSE is only

$$\frac{n - p - 1}{n + p + 1} < 1$$

times the expected validation-set MSE. A fitted model essentially always fits its own training data better than fresh data, and the gap grows when the training set is small or the parameter count is large. For most other procedures (e.g., logistic regression) no simple formula exists, so cross-validation substitutes numerical computation for theoretical analysis.

The lecture presents two concrete schemes:

1. **Leave-one-out cross-validation (LOOCV)**: hold out one example, train on all the rest, test on the held-out example, and repeat for every example in the dataset, averaging the results. Every example gets used for both training and testing — just never at the same time. In the taxonomy of cross-validation methods, this is the special case $p = 1$ of *leave-p-out* cross-validation, which exhaustively

tries all ways of splitting $n$ observations into a validation set of size $p$ and a training set — requiring $C_p^n$ fits. That combinatorial cost explodes quickly: with $n = 100$ and $p = 30$, $C_{30}^{100} \approx 3 \times 10^{25}$, which is computationally infeasible; LOOCV survives because $p = 1$ means only $n$ fits. (Its mechanics resemble the jackknife, with the statistic computed on the left-out sample.) A related variant, *leave-pair-out* cross-validation, has been recommended as a nearly unbiased estimator of the area under the ROC curve for binary classifiers — directly relevant to a two-class problem like survival prediction.

2. **Repeated random subsampling**: repeatedly split the data at random into a training set and a test set, evaluate on each split, and average over many repetitions. It is cheaper per trial than leave-one-out, and with enough repetitions yields a reliable estimate.

Either way, the principle is identical: an honest estimate of performance must come from data the model did not learn from.

## From a single holdout to systematic evaluation: cross-validation

Holding out one fixed test set gives an estimate of performance, but it has two weaknesses: some data never contributes to training, and a single split can be lucky or unlucky. **Cross-validation** — also called *rotation estimation* or *out-of-sample testing* — addresses this by using different portions of the data to train and test on different iterations. One round partitions the sample into complementary subsets, performs the analysis on one subset (the training set), and validates on the other (the validation/testing set). Because any single round is variable, most methods perform *multiple rounds* with different partitions and combine (e.g., average) the results into a single estimate of predictive performance.

Why go to this trouble? When you fit a model, the fitting process optimizes its parameters to fit the *training* data as well as possible. An independent sample from the same population will generally not be fit as well, and the gap grows when the training set is small or the model has many parameters. In linear regression this gap can even be quantified: under mild assumptions, the expected training-set mean squared error is $\frac{n-p-1}{n+p+1} < 1$ times the expected validation-set MSE (where $n$ is the number of observations and $p$ the number of covariates). So evaluating on training data yields an optimistically biased **in-sample estimate**, whereas cross-validation yields an **out-of-sample estimate**. For procedures like logistic regression there is no closed-form correction factor, so cross-validation is the generally applicable way to predict performance on unavailable data — numerically instead of theoretically. Its goals are to test the model's ability to predict unseen data, to flag problems like overfitting and selection bias, and to assess the stability of fitted parameters.

Cross-validation methods divide into **exhaustive** (learn and test on *all possible* ways to divide the sample) and **non-exhaustive** (use a sampling of splits). The two harnesses built in this lecture are one of each.

## Leave-one-out testing: the exhaustive extreme

**Leave-one-out** testing holds out each example in turn: for index $i$, that single example becomes the test case and the model is trained on everything else. In the code, `leaveOneOut(examples, method, toPrint=True)` initializes four counters (`truePos`, `falsePos`, `trueNeg`, `falseNeg`) to zero, then loops over `range(len(examples))`. For each `i`, it sets `testCase = examples[i]` and builds the training data by slicing:

```
trainingData = examples[0:i] + examples[i+1:]
```

— everything before example $i$ glued to everything after it, i.e., all examples except the held-out one. It then calls `method(trainingData, [testCase])`, which returns four numbers that are accumulated into the running totals (`truePos += results[0]`, and so on). After the loop, `getStats` prints summary statistics if requested, and the four totals are returned.

This is the $p = 1$ special case of **leave-$p$-out cross-validation (LpO CV)**, an exhaustive method that uses $p$ observations for validation and the rest for training, repeated over *all* ways to cut the sample. Full LpO requires training and validating the model $C_p^n = \binom{n}{p}$ times, which quickly becomes computationally

infeasible: with $n = 100$ and $p = 30$, $\binom{100}{30} \approx 3 \times 10^{25}$. Leave-one-out stays feasible because $\binom{n}{1} = n$: exactly $n$ model builds. (A related variant, leave-pair-out with $p = 2$, has been recommended as a nearly unbiased estimator of the area under the ROC curve for binary classifiers. LOOCV also resembles the jackknife, but with cross-validation the statistic is computed *on the left-out sample*, whereas jackknifing computes it differently.)

The appeal of leave-one-out is that **every example gets used for both training and testing — no data is wasted**. The cost is exactly that $n$-fold retraining: with $n$ examples you build and test a model $n$ times, which can get expensive.

## Repeated random subsampling: the non-exhaustive alternative

The second harness trades exhaustiveness for speed. The helper `split80_20` produces one fresh random split per call:

```
sampleIndices = random.sample(range(len(examples)), len(examples)//5)
```

`random.sample` picks a random set of indices covering one fifth of the examples (~20%). The function then loops over all examples: index $i$ goes into the `testSet` if $i$ is in `sampleIndices`, otherwise into the `trainingSet`, and both are returned — a fresh random 80/20 partition each time.

The driver `randomSplits(examples, method, numSplits, toPrint=True)` runs this repeatedly. It initializes the same four counters, and — importantly — calls `random.seed(0)` first. Fixing the seed pins the random number generator so every run of the experiment produces the same sequence of splits; this is what makes the experiment **reproducible**: re-running it must give the same answer. Then for each of the `numSplits` iterations it obtains a new `(trainingSet, testSet)` pair and executes the highlighted line `results = method(trainingSet, testSet)`, accumulating the four counts. At the end it reports the *average per split* — passing `truePos/numSplits`, `falsePos/numSplits`, `trueNeg/numSplits`, `falseNeg/numSplits` to `getStats` — and returns those averages. This is the non-exhaustive counterpart to leave-one-out: rather than all $n$ rotations, we sample `numSplits` random partitions and average, which is precisely the "multiple rounds, combined results" pattern cross-validation prescribes for reducing variability.

## The classifier under test: KNN on the Titanic data

Both harnesses expect a `method` with the interface `method(trainingSet, testSet) -> (truePos, falsePos, trueNeg, falseNeg)`. The classifier plugged in is `KNearestClassify(training, testSet, label, k)`, whose docstring states exactly that contract: given lists of examples and an integer $k$, it predicts whether each test example has the label and returns the counts of true positives, false positives, true negatives, and false negatives. To adapt it to the harnesses' two-argument signature, the label and $k$ are fixed with a lambda:

```
knn = lambda training, testSet: KNearestClassify(training, testSet, 'Survived', 3)
```

so the experiment predicts the `'Survived'` label on the Titanic data using the three nearest neighbors.

What is this classifier doing? The **k-nearest neighbors algorithm (k-NN)** is a non-parametric supervised learning method that assigns weight only to the $k$ nearest neighbors of an entity when deciding about it. For classification, the output is a class membership decided by **plurality vote** among those $k$ neighbors; $k$ is typically small, and the $k = 1$ case is simply the nearest neighbor algorithm. It originated with Evelyn Fix and Joseph Hodges in 1951 and was later expanded by Thomas Cover. Two properties matter for understanding how it interacts with the evaluation harnesses:

- **There is no real training step.** The "training phase" consists only of storing the feature vectors and labels; all computation is deferred until function evaluation (a lazy, locally-approximated function). This is why the harnesses can afford to call the method $n$ times or `numSplits` times — though each call still pays full distance computations at prediction time.
- **It depends entirely on distances.** With continuous variables, Euclidean distance is the common metric (discrete data may use overlap/Hamming distance; gene-expression work has used Pearson and

Spearman correlations). If features come in different physical units or vastly different scales, feature-wise normalization of the training data can greatly improve accuracy, and noisy or irrelevant features can severely degrade it.

The figure above shows the vote at work: a test point (green) is classified red triangles when $k = 3$ (2 triangles vs. 1 square nearby) but blue squares when $k = 5$ (3 squares vs. 2 triangles). The choice of $k$ therefore matters: larger $k$ dampens the effect of noise but makes class boundaries less distinct, and a good value is selected by heuristic techniques (hyperparameter optimization). Two known failure modes are worth noting: with skewed class distributions, majority voting lets the more frequent class dominate predictions (mitigated by weighting each neighbor's vote by inverse distance, proportional to $1/d$, or by abstracting the data, e.g., applying k-NN over self-organizing map nodes); and accuracy can be improved by learning the metric itself (e.g., large margin nearest neighbor, neighborhood components analysis).

## Reading the results: two protocols, one conclusion

Running both harnesses with the same classifier and data (`numSplits = 10` for the random-splits experiment):

| Protocol | Accuracy | Sensitivity | Specificity | PPV |
|---|---|---|---|---|
| Average of 10 80/20 splits, KNN ($k = 3$) | 0.766 | 0.67 | 0.836 | 0.747 |
| Average of LOO testing, KNN ($k = 3$) | 0.769 | 0.663 | 0.842 | 0.743 |

Two takeaways. First, the headline: these numbers are **considerably better than 62%**, the earlier baseline obtained by simply predicting the most common outcome. KNN is genuinely extracting signal from the features — adding real value beyond the trivial baseline. Second, there is **not much difference between the experiments**: accuracy around 0.77, sensitivity around 0.67, specificity around 0.84 under both protocols. That agreement is itself informative. When two quite different evaluation protocols — one exhaustive, one randomly sampled — converge on essentially the same picture, we gain confidence that the result reflects the model's real performance rather than an artifact of how the data happened to be split. That is exactly the assurance cross-validation is designed to provide.